



微算機系統 Fall 2026

MICROPROCESSOR SYSTEMS

Instructor : Yen-Lin Chen(陳彥霖)

Dept. Computer Science and Information Engineering

National Taipei University of Technology

ylchen@ntut.edu.tw



計算機組織相關內容參考 - Pipeline



Pipeline發展趨勢

- 早期(Pentium 4)：不斷加深pipeline(>20)，提高時脈
 - 缺點:過度依賴branch prediction，導致效能不佳。
 - 現今已用其他方式提升IPC。
- 現在：
 - 更強大的亂序執行：
 - 一系列硬體單元實現，如：指令視窗、預約站、重新命名、ROB等，大幅提升IPC。
 - 超純量(Superscalar)：
 - 讓每個cycle可以issue更多指令（現代高效能核心多為4~8 way，如 Apple M 系列 performance core 可達 8-way）
 - 利用神經網路等更強大的工具來做分支預測：
 - 比起傳統的1-bit 2-bit，神經預測可以一次看上百條分支紀錄來做預測，進而處理更複雜的情況及更高的準確度。

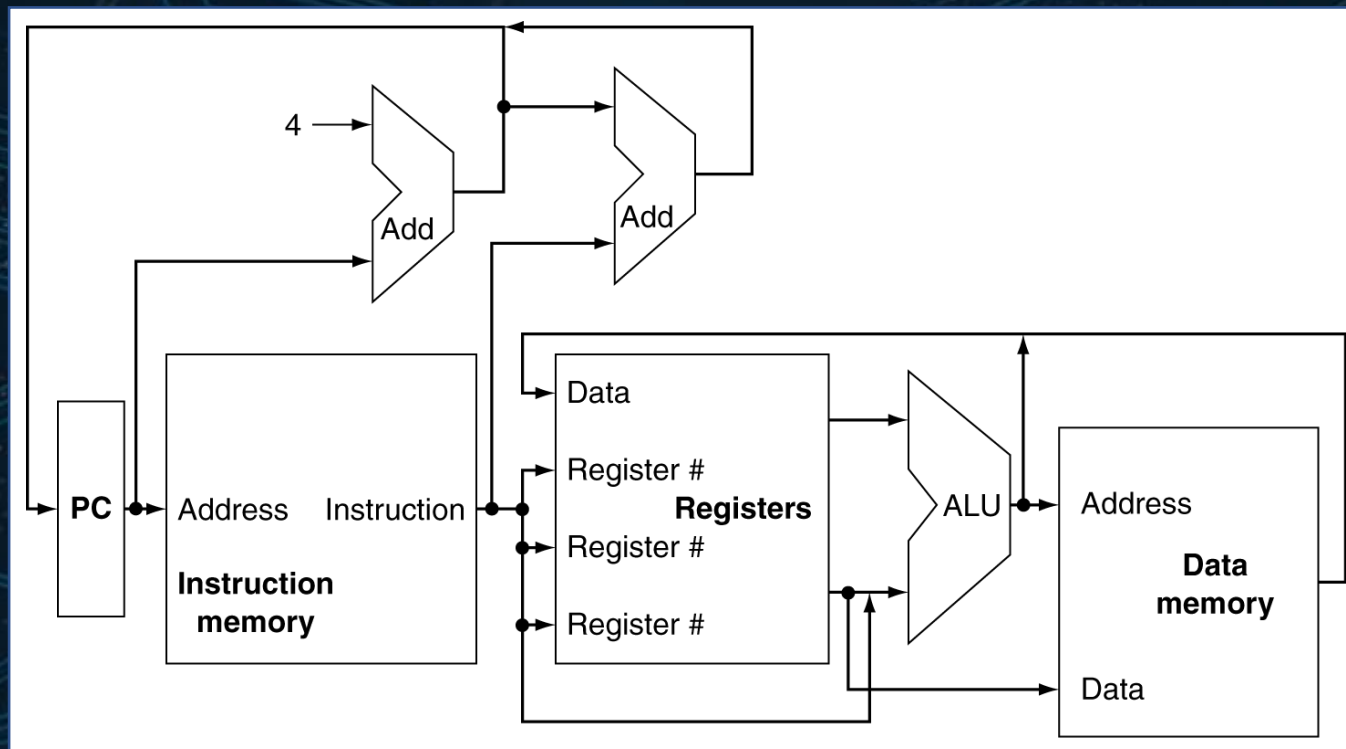


Introduction

- CPU performance factors
 - Instruction count
 - Determined by ISA and compiler
 - CPI and Cycle time
 - Determined by CPU hardware
- We will examine two MIPS implementations
 - A simplified version
 - A more realistic **pipelined** version
- Simple subset, shows most aspects
 - Memory reference: **lw, sw**
 - Arithmetic/logical: **add, sub, and, or, slt**



CPU Overview

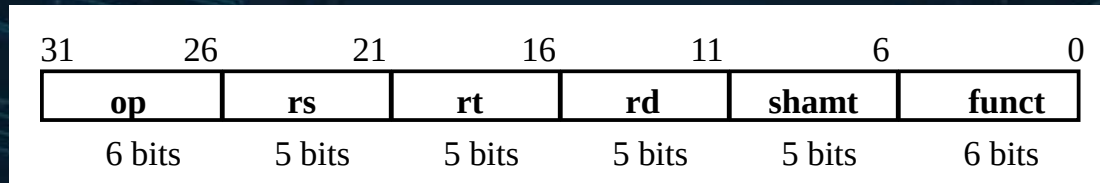




Analyze Instruction Set

- R-type MIPS instructions are 32 bits long:

- **R-type:**



- These fields are:
 - **op**: operation of the instruction
 - **rs, rt, rd**: source and destination register
 - **shamt**: shift amount
 - **funct**: selects variant of the “op” field



Example: R-type MIPS Subset

R-Type:

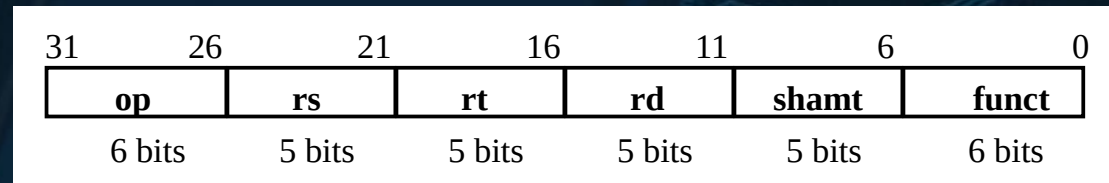
add rd, rs, rt

sub rd, rs, rt

and rd, rs, rt

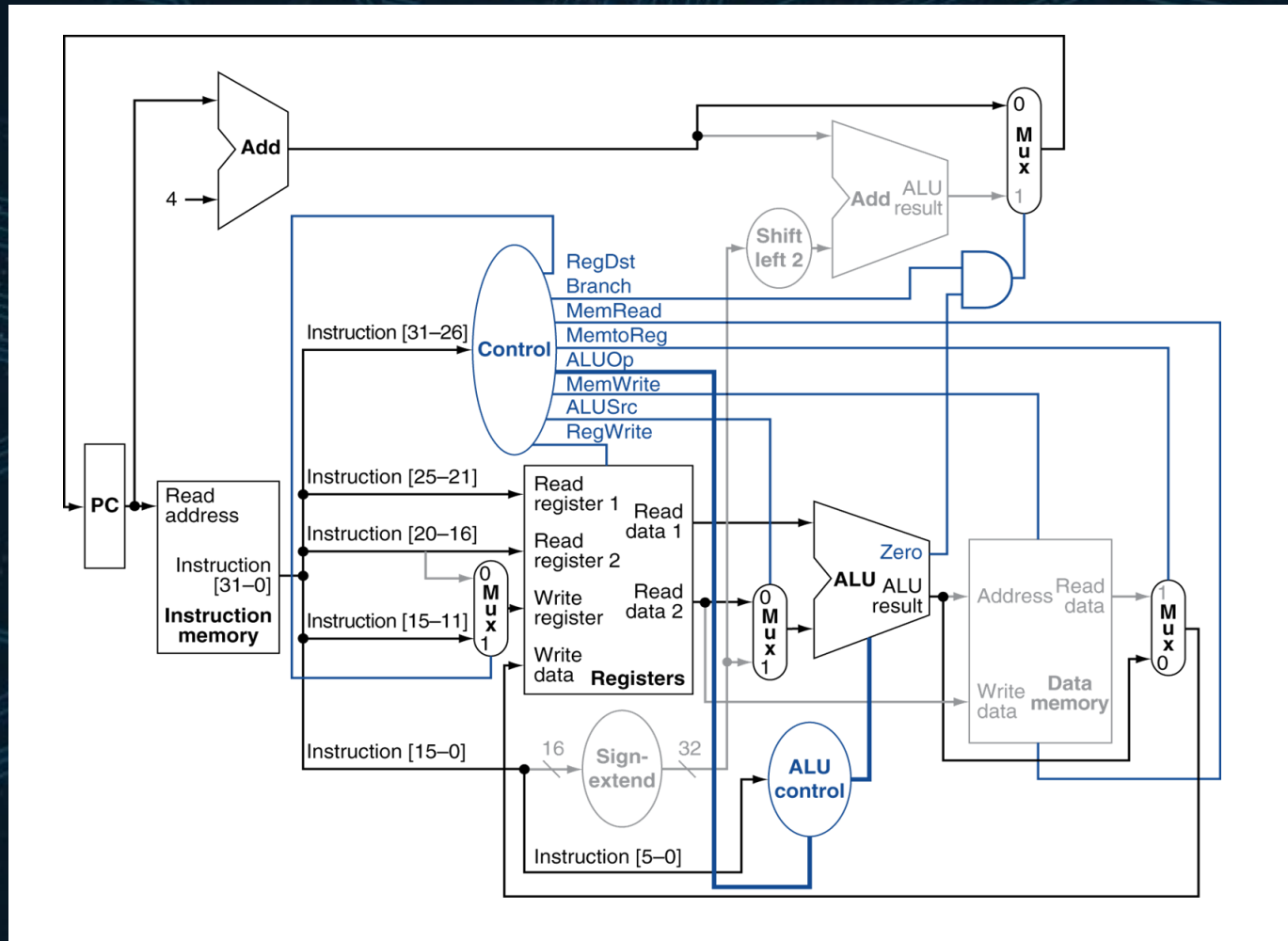
or rd, rs, rt

slt rd, rs, rt





R-Type Instruction



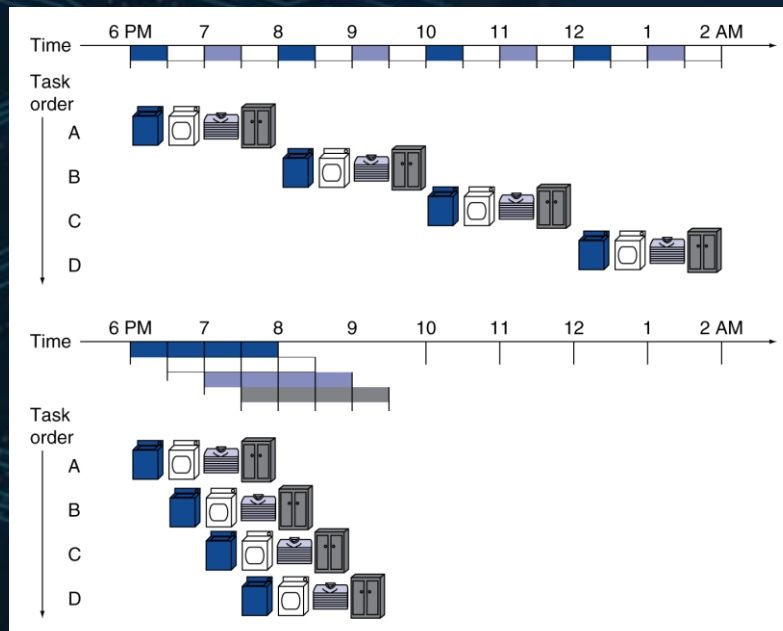


MIPS Pipeline

- Five stages, one step per stage
 1. **IF**: Instruction fetch from memory
 2. **ID**: Instruction decode & register read
 3. **EX**: Execute operation or calculate address
 4. **MEM**: Access memory operand
 5. **WB**: Write result back to register

Pipelining Analogy

- Pipelined laundry: overlapping execution
- Parallelism improves performance



- Four loads:
 - Speedup

$$= 8 / 3.5 = 2.3$$
- Non-stop:
 - Speedup

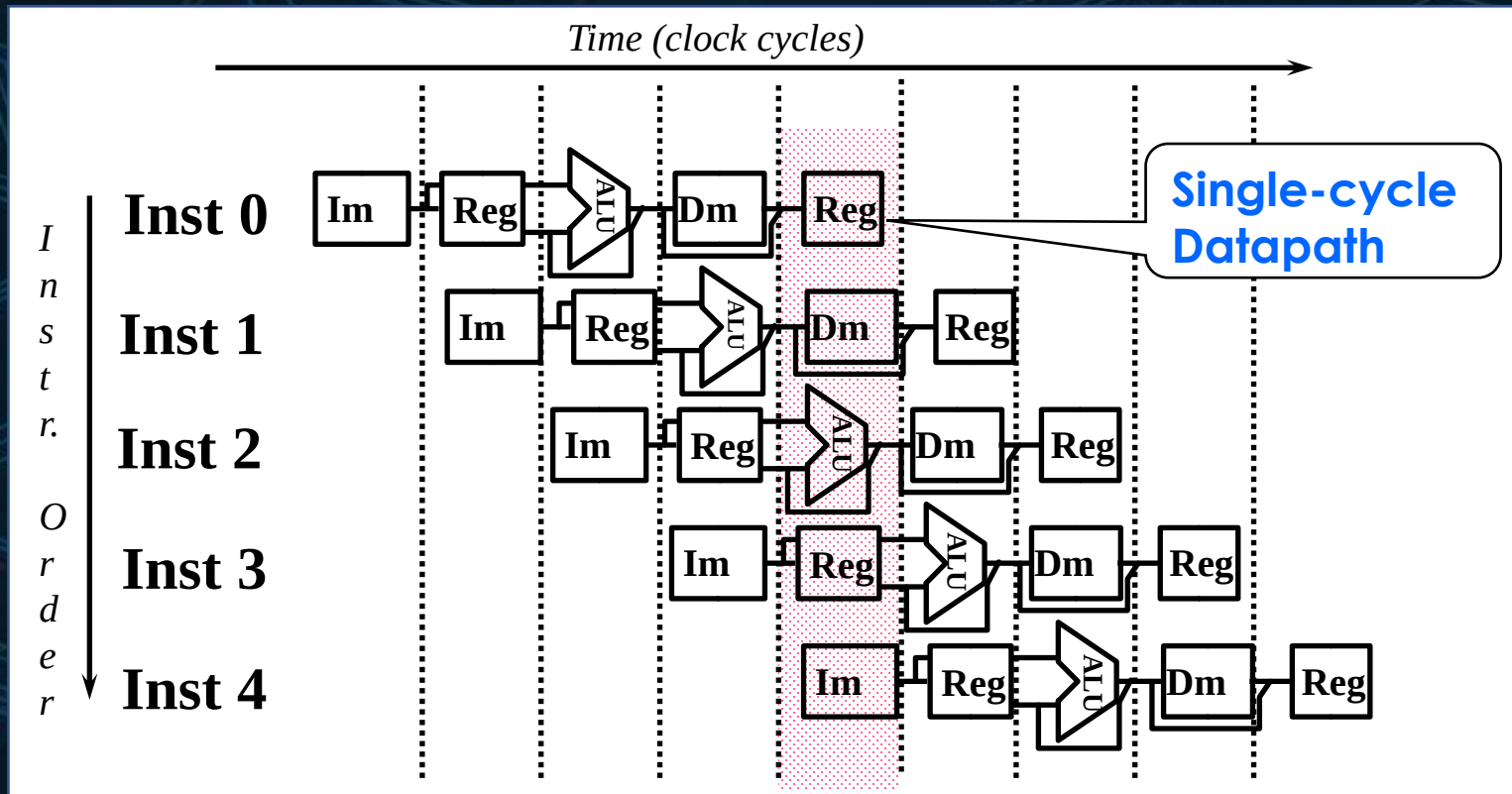
$$= 2n / (0.5n + 1.5) \approx 4$$

$$= \text{number of stages}$$

When $n \rightarrow \infty$
 Then Speedup $\approx 2n / 0.5n = 4$



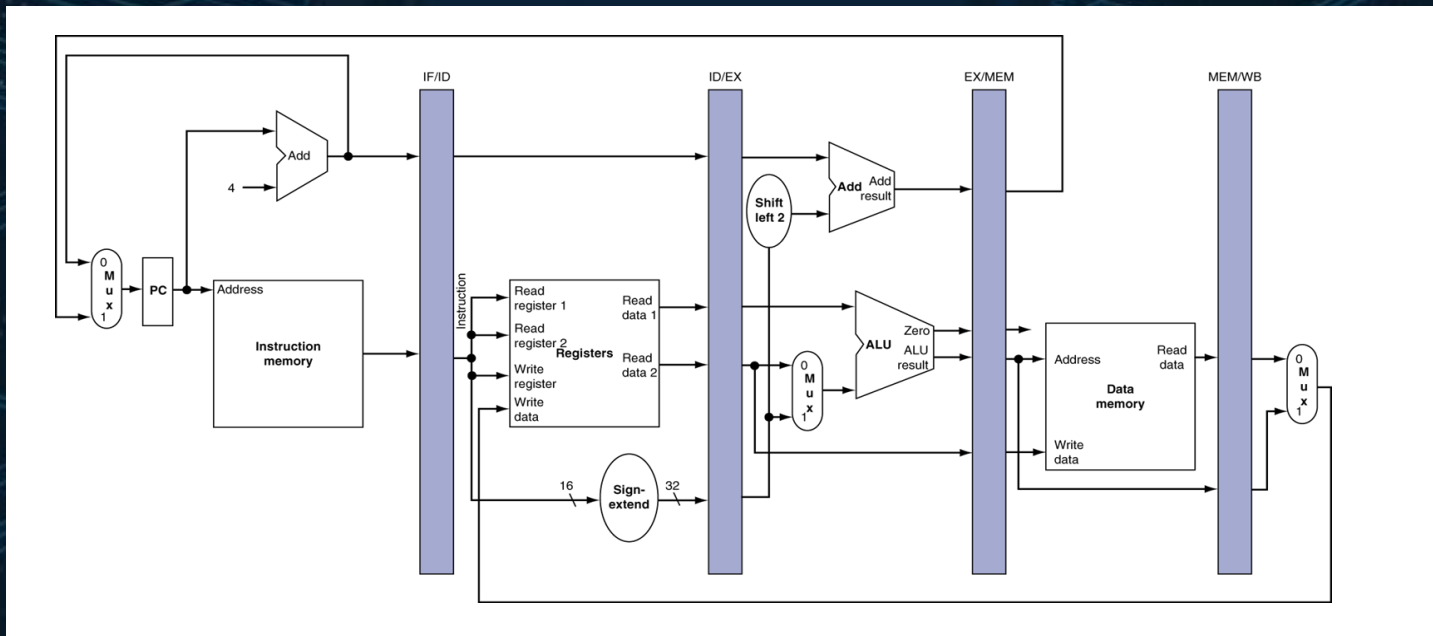
Why Pipeline?





Pipeline registers

- Need registers between stages
 - To hold information produced in previous cycle





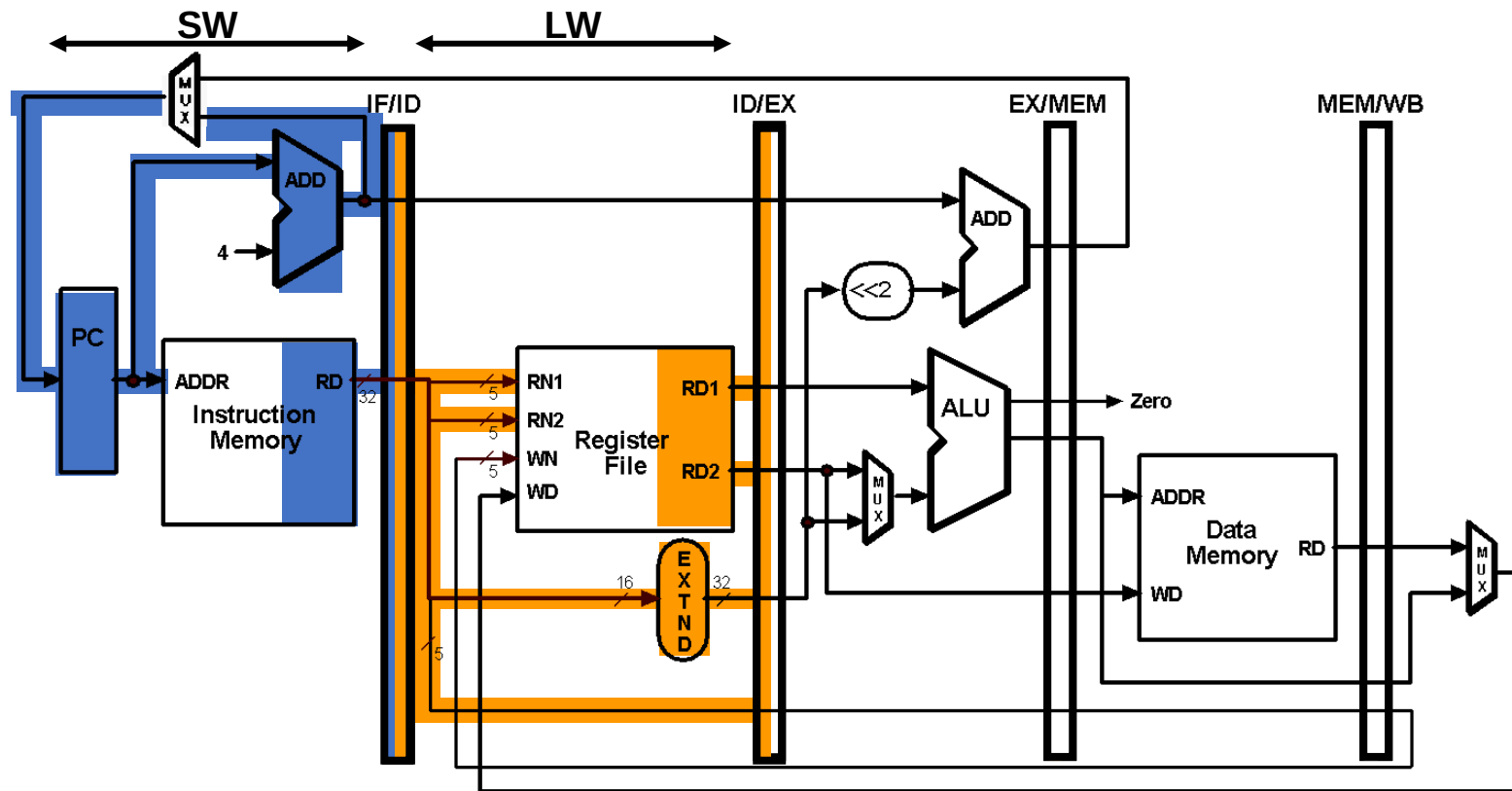
The diagram illustrates the internal structure of a processor, showing the flow of data and control signals through five stages: LW, IF/ID, ID/EX, EX/MEM, and MEM/WB.

Key Components and Connections:

- PC (Program Counter):** Provides the address for Instruction Memory.
- Instruction Memory:** Outputs a 32-bit instruction to the IF/ID stage. It has an **ADDR** (Address) input and an **RD** (Read Data) output.
- IF/ID Stage:** The instruction is split into fields: **OP** (4 bits), **RD** (5 bits), **RN1** (5 bits), **RN2** (5 bits), **WN** (5 bits), and **WD** (5 bits).
- ID/EX Stage:** The **RD** field is used to select a register from the **Register File**. The **Register File** outputs 32-bit values for **RN1**, **RN2**, **WN**, and **WD**. The **WD** field is also used to select a register for writing back.
- EX/MEM Stage:** The **ALU** performs operations on **RN1** and **RN2** based on the **OP** field. The **ALU** output is zero if the result is zero. The **ALU** also performs a shift operation (**SHL**) on **RN1** by the value in **WN**. The **ALU** output is used to calculate the **PC** for the next instruction.
- MEM/WB Stage:** The **ALU** output is used to calculate the **PC** for the next instruction. The **ALU** output is also used to calculate the **PC** for the next instruction.
- Register File:** A central component that stores register values. It has inputs for **RN1**, **RN2**, **WN**, and **WD**, and outputs for **RN1**, **RN2**, **WN**, and **WD**.
- ALU (Arithmetic Logic Unit):** Performs operations on **RN1** and **RN2** based on the **OP** field. It also performs a shift operation (**SHL**) on **RN1** by the value in **WN**.
- Data Memory:** Outputs a 32-bit value to the **MEM/WB** stage. It has an **ADDR** (Address) input and an **RD** (Read Data) output.
- PC (Program Counter):** Provides the address for Instruction Memory.

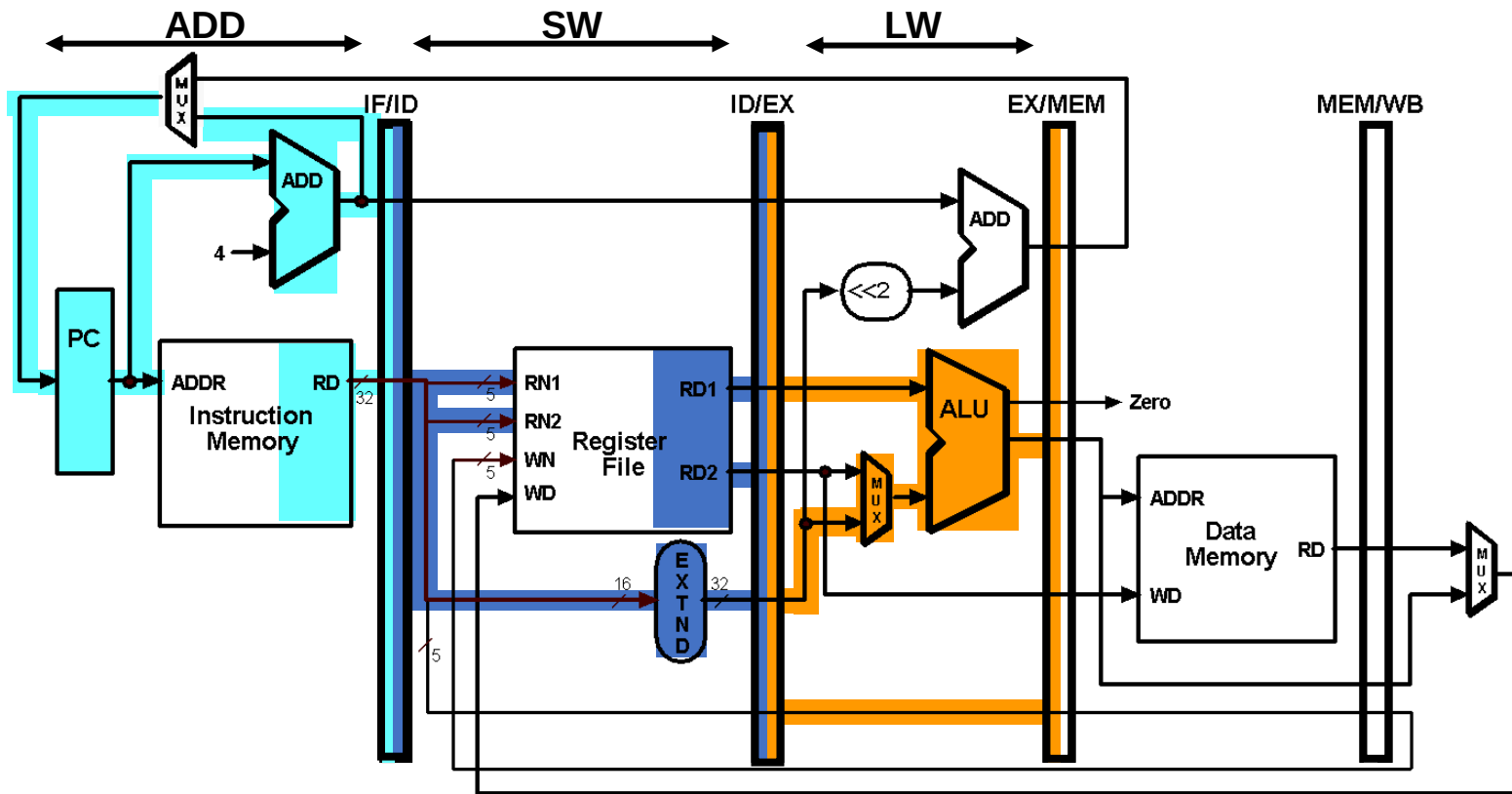


Single-Clock-Cycle Diagram: Clock Cycle 2



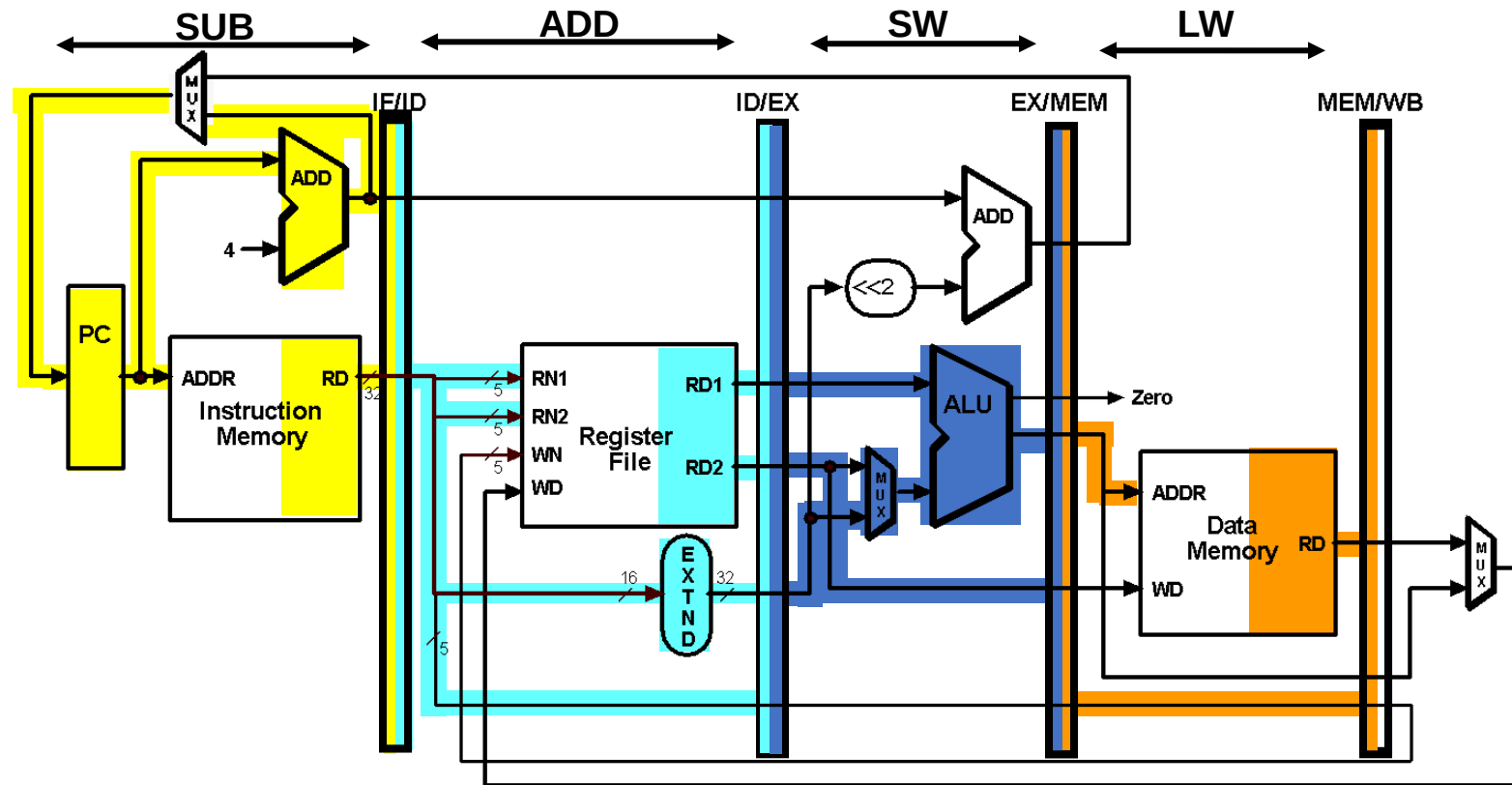


Single-Clock-Cycle Diagram: Clock Cycle 3



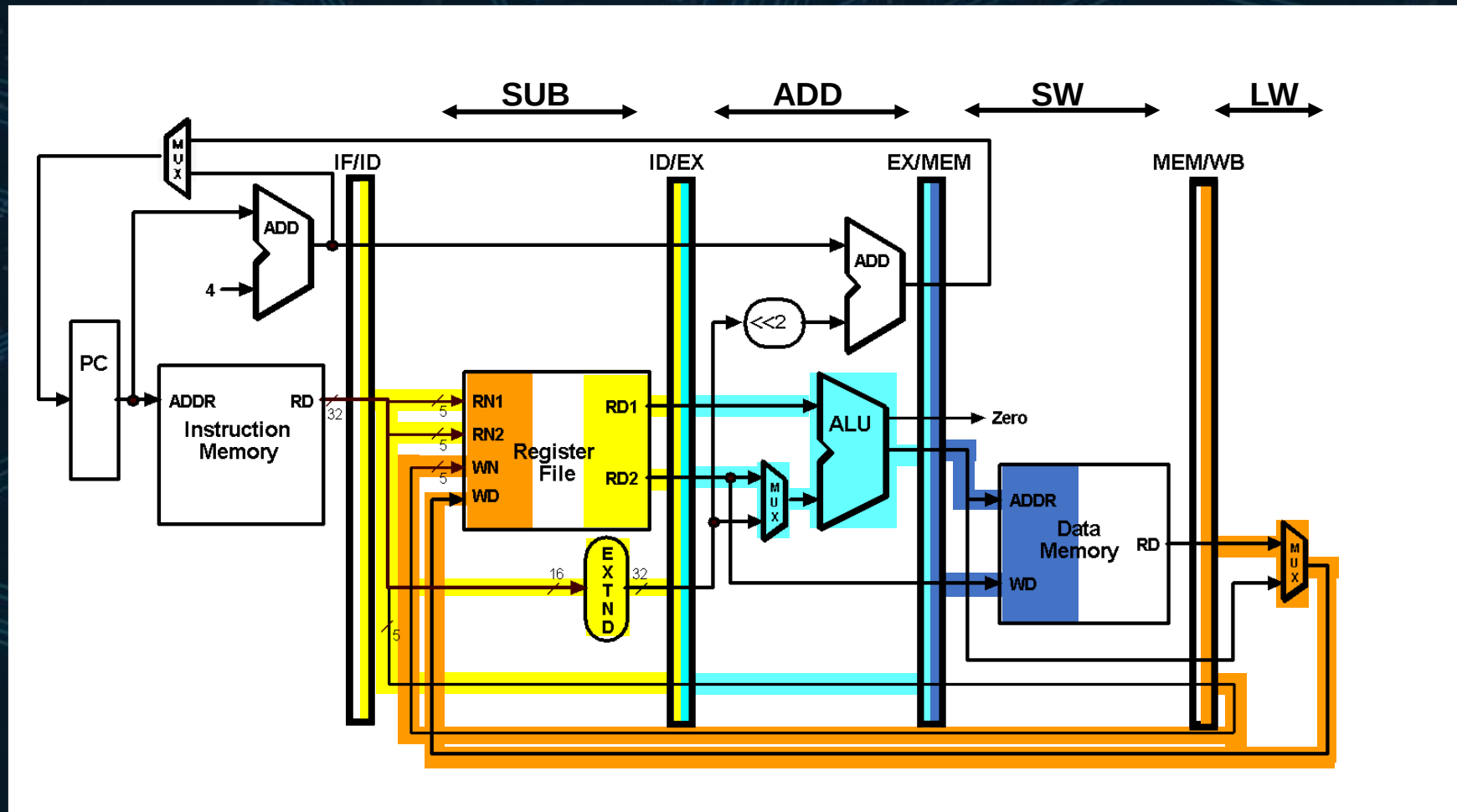


Single-Clock-Cycle Diagram: Clock Cycle 4



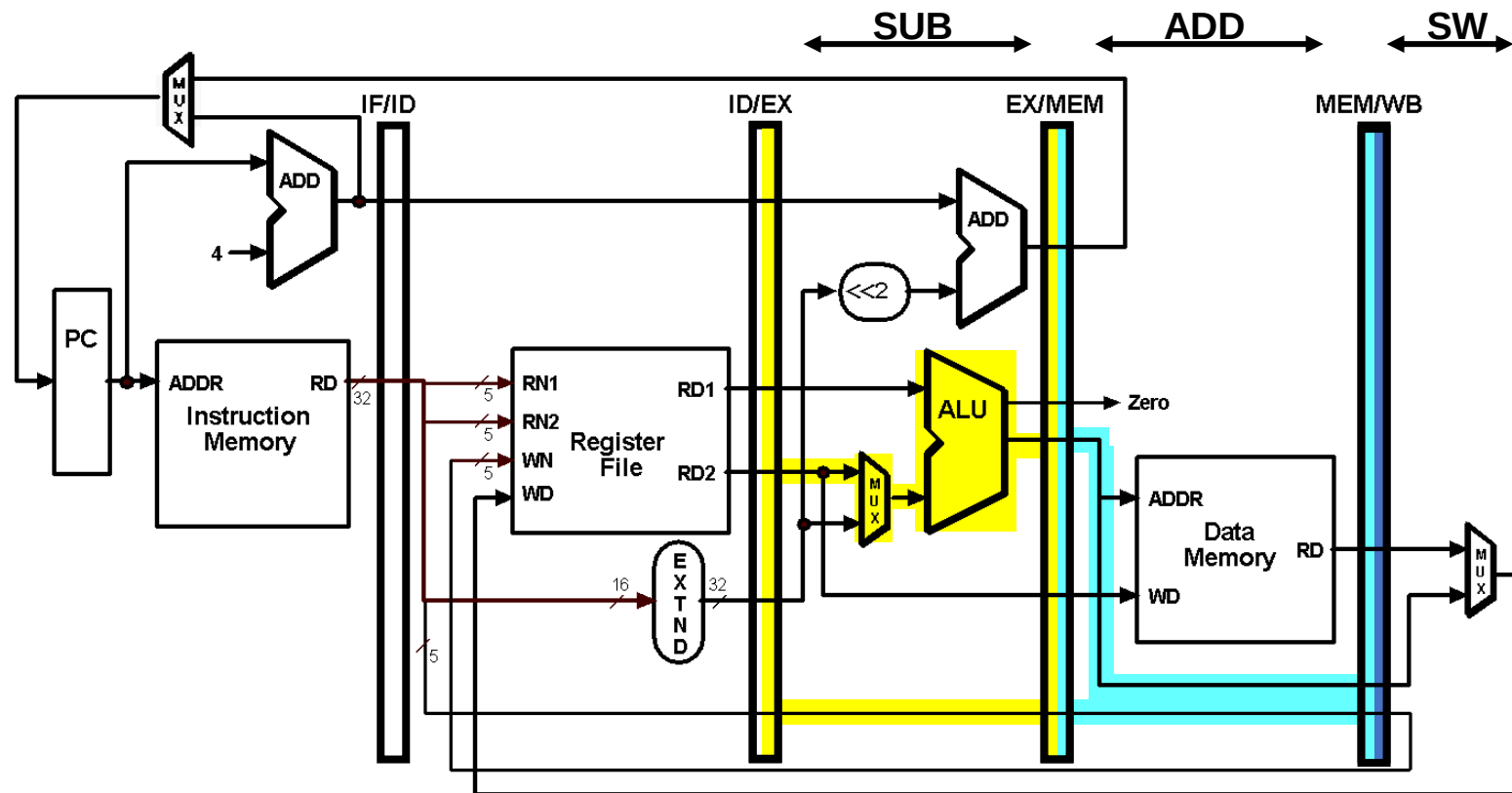


Single-Clock-Cycle Diagram: Clock Cycle 5



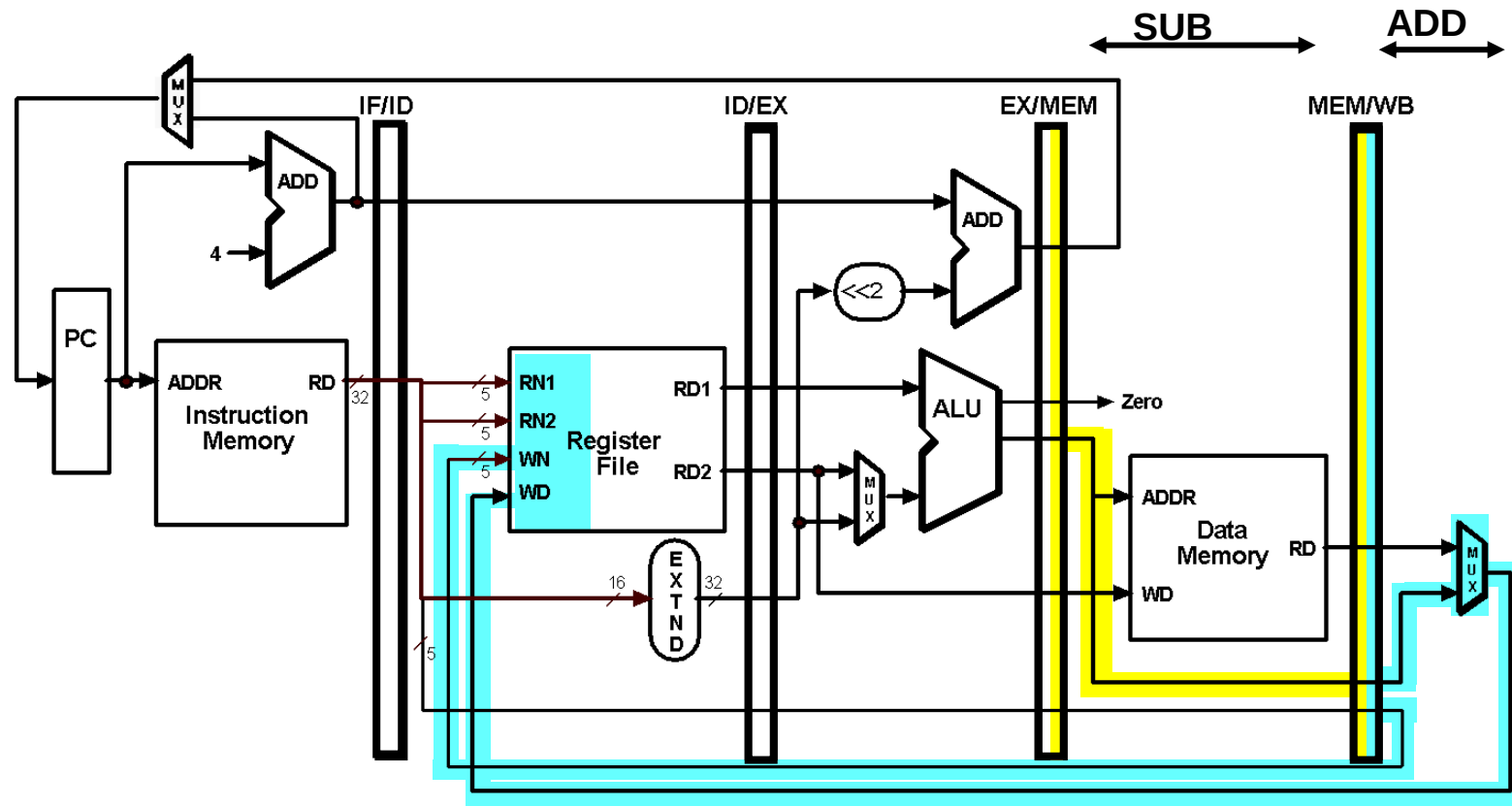


Single-Clock-Cycle Diagram: Clock Cycle 6



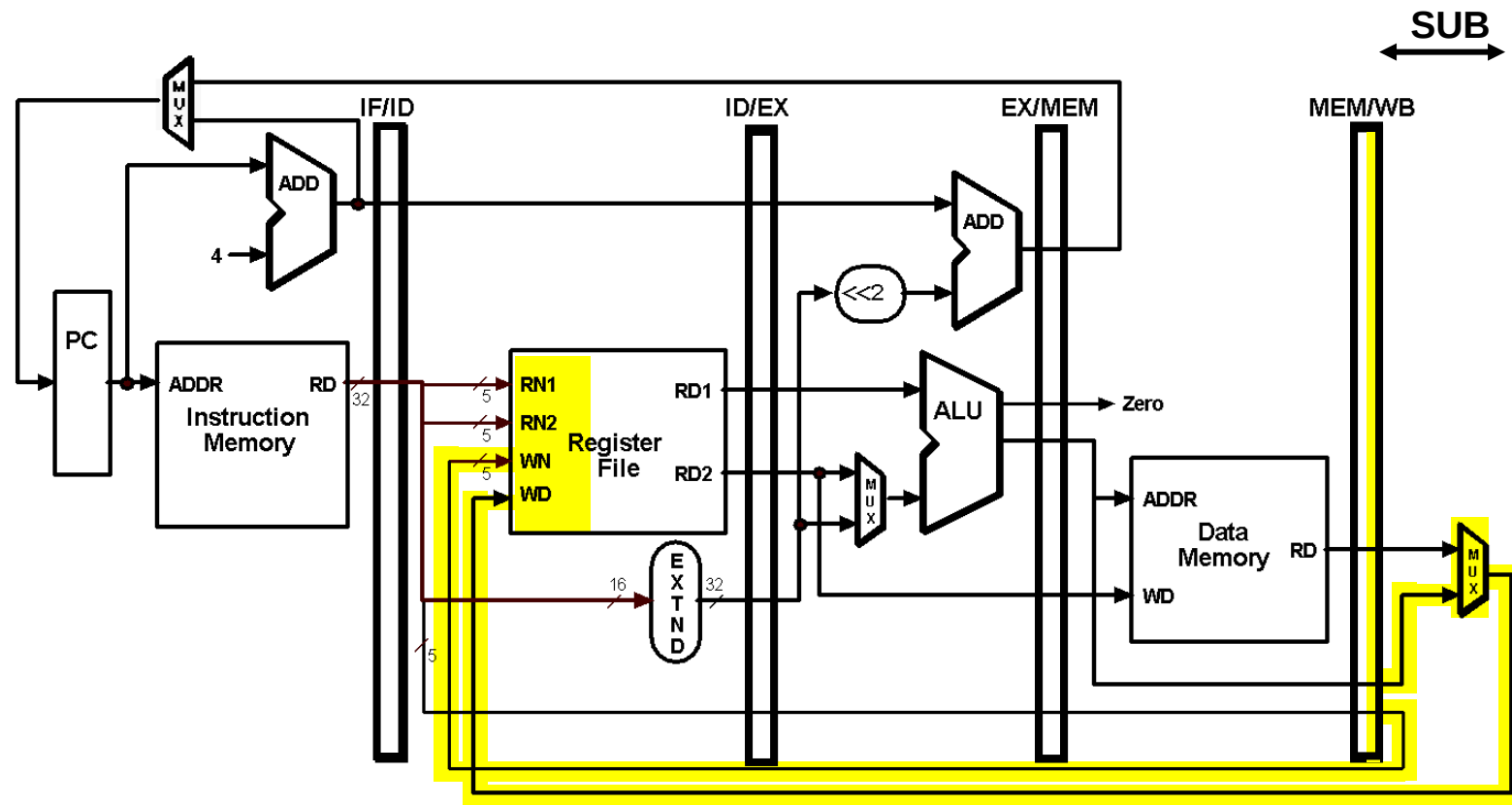


The diagram illustrates the internal structure of a 5-stage MIPS processor. The stages are labeled at the top: IF/ID, ID/EX, EX/MEM, and MEM/WB. The PC (Program Counter) feeds into the Instruction Memory (ADDR, RD) and the ADD instruction's 4-bit constant. The Instruction Memory outputs a 32-bit RD to the Register File (RN1, RN2, WN, WD). The Register File outputs 5-bit RD1 and RD2 to the ALU and a 16-bit EXTND to the EX/MEM stage. The ALU performs operations (SUB or ADD) on the 32-bit RD1 and RD2, producing a Zero flag. The EX/MEM stage outputs a 32-bit RD to the Data Memory (ADDR, RD). The Data Memory outputs a 32-bit RD to the MEM/WB stage. The MEM/WB stage outputs a 32-bit RD to the Register File. The diagram also shows the flow of data between the stages and the final write-back to the Register File. The SUB and ADD instructions are highlighted with yellow and cyan paths, respectively.





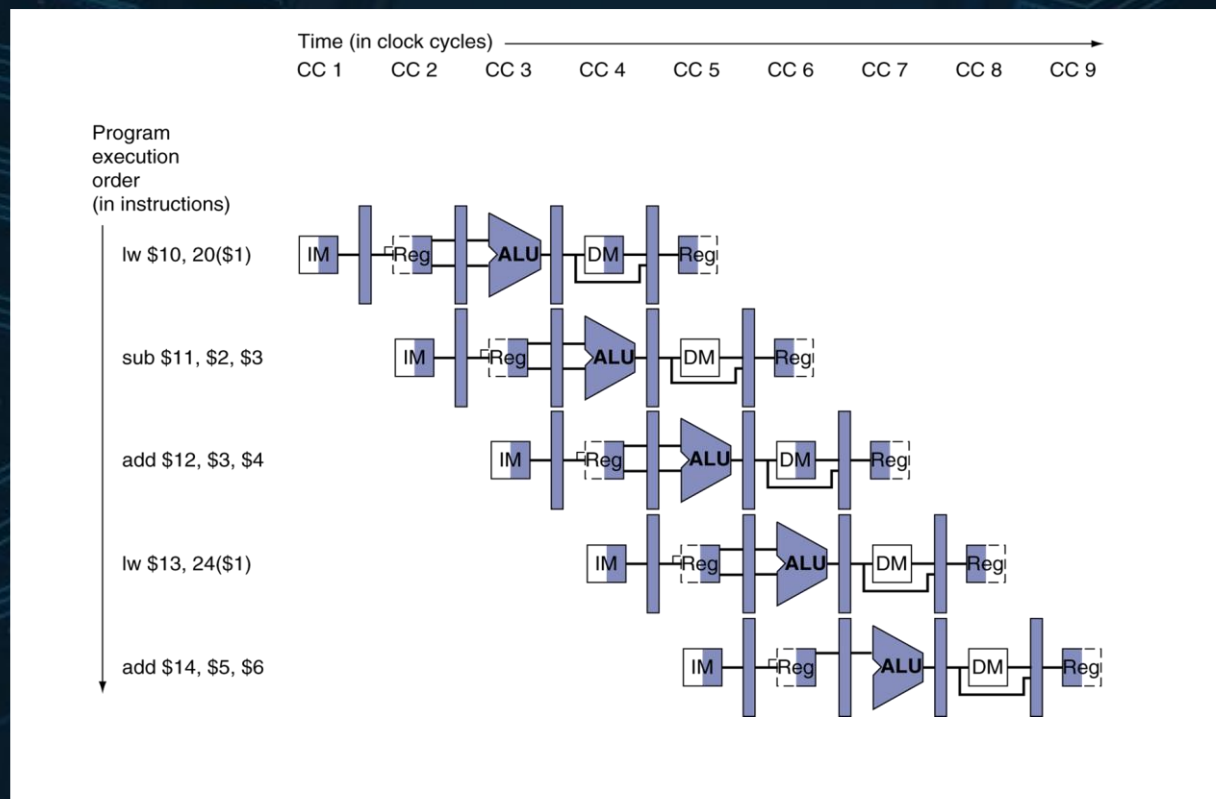
Single-Clock-Cycle Diagram: Clock Cycle 8





Alternative View – Multi-Cycle Pipeline Diagram

- Form showing resource usage





Pipeline Hazards

- Pipeline Hazards:
 - **Structural hazards**: attempt to use the same resource in two different ways at the same time
 - **Data hazards**: attempt to use item before ready
 - Instruction depends on result of prior instruction still in the pipeline
 - **Control hazards**: attempt to make decision before condition is evaluated
 - Branch instructions
- Can always resolve hazards by **waiting (stalls)**
 - pipeline control must detect the hazard
 - take action (or delay/stall action) to resolve hazards



Data Hazards in ALU Instructions

- Consider this sequence:

sub \$2, \$1, \$3

and \$12, \$2, \$5

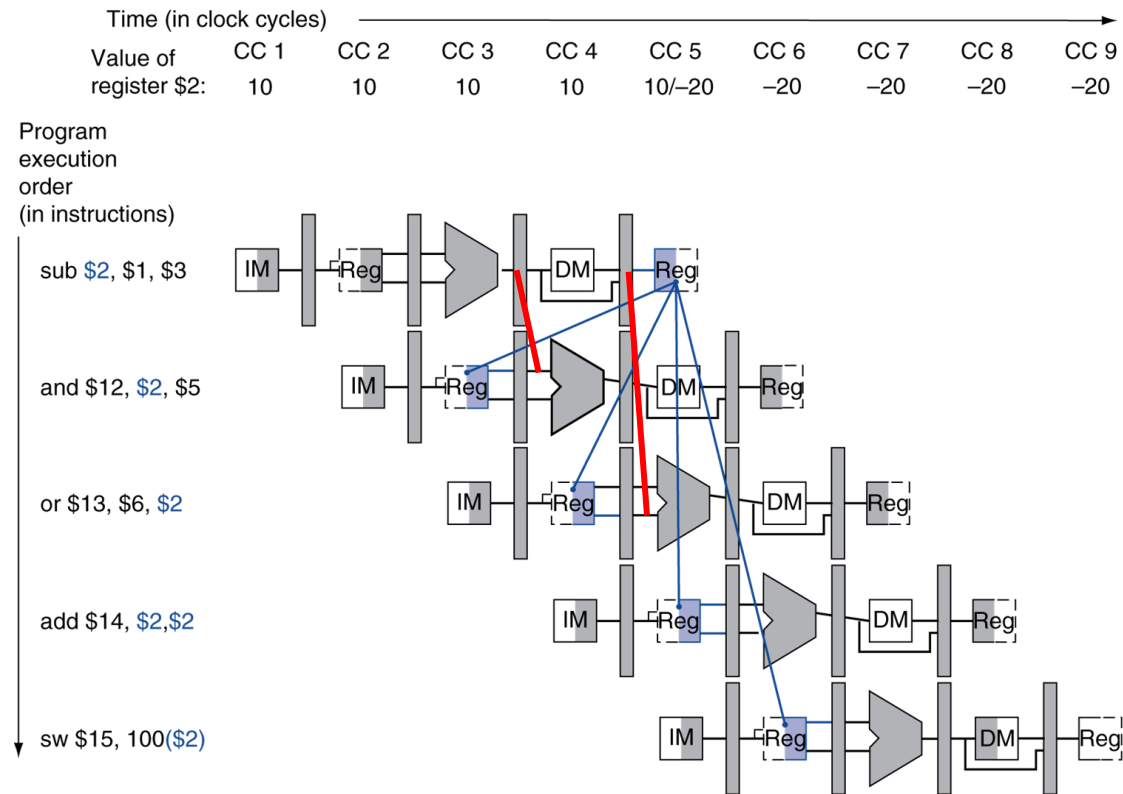
or \$13, \$6, \$2

add \$14, \$2, \$2

sw \$15, 100(\$2)

- We can resolve hazards with forwarding
 - How do we detect when to forward?

Dependencies & Forwarding





Hardware Solution: Forwarding

- Idea: use **intermediate data**, **do not wait for result** to be finally written to the destination register.
- Two steps:
 1. Detect data hazard
 2. Forward intermediate data to resolve hazard



Detecting the Need to Forward

- Pass register numbers along pipeline
 - e.g., **ID/EX.RegisterRs** = register number for Rs sitting in **ID/EX** pipeline register
- ALU operand register numbers in EX stage are given by
 - **ID/EX.RegisterRs**, **ID/EX.RegisterRt**
- Data hazards when
 - 1a. $\text{EX/MEM.RegisterRd} = \text{ID/EX.RegisterRs}$
 - 1b. $\text{EX/MEM.RegisterRd} = \text{ID/EX.RegisterRt}$
 - 2a. $\text{MEM/WB.RegisterRd} = \text{ID/EX.RegisterRs}$
 - 2b. $\text{MEM/WB.RegisterRd} = \text{ID/EX.RegisterRt}$

Fwd from EX/MEM
pipeline reg

Fwd from MEM/WB
pipeline reg



Hazard Detection

- Hazard conditions:

1a. $EX/MEM.RegisterRd = ID/EX.RegisterRs$

1b. $EX/MEM.RegisterRd = ID/EX.RegisterRt$

2a. $MEM/WB.RegisterRd = ID/EX.RegisterRs$

2b. $MEM/WB.RegisterRd = ID/EX.RegisterRt$

- E.g., in the earlier example, first hazard between sub \$2, \$1, \$3 and and \$12, \$2, \$5 is detected when the and is in EX stage and the sub is in MEM stage because

- $EX/MEM.RegisterRd = ID/EX.RegisterRs = \2 (1a)

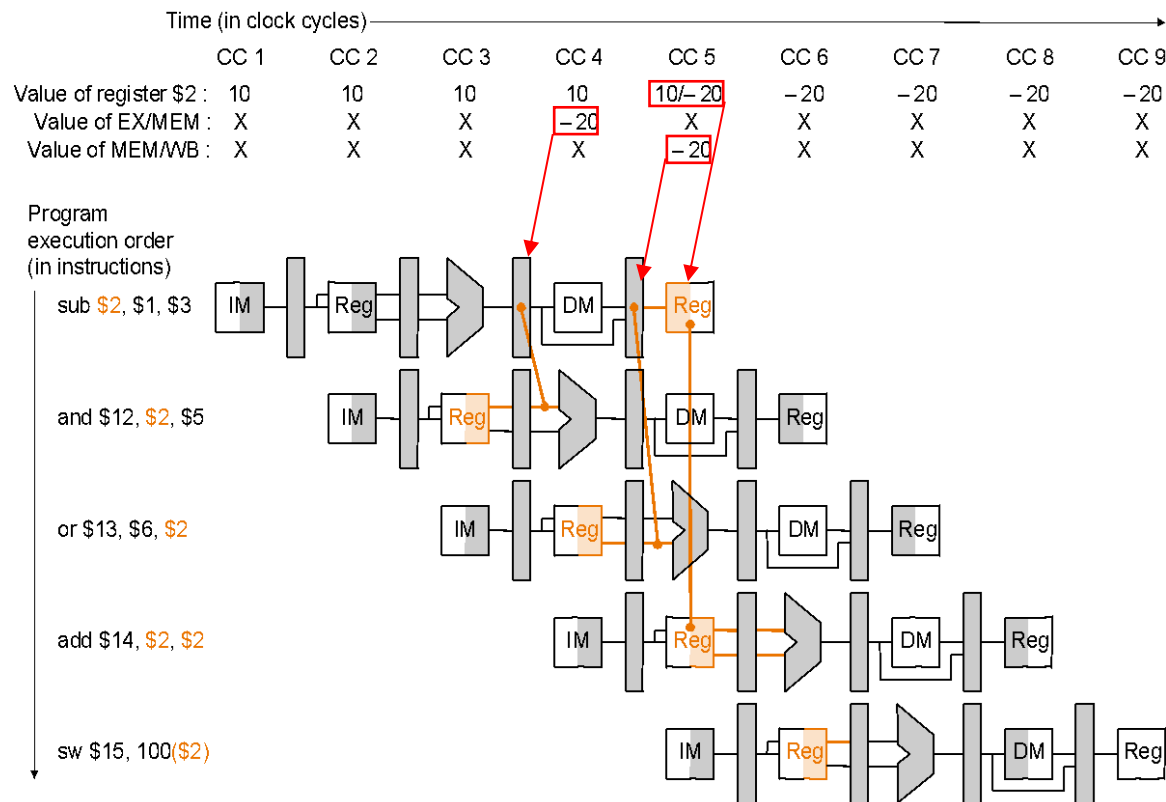
- Whether to forward also depends on:

- if the instruction in EX/MEM (or MEM/WB) is going to write a register – if not, no need to forward, even if there is register number match as in conditions above
 - Only if $EX/MEM.RegWrite, MEM/WB.RegWrite$
- if the destination register of the later instruction is \$0 – in which case there is no need to forward value (\$0 is always 0 and never overwritten)
 - Only if $EX/MEM.RegisterRd \neq 0, MEM/WB.RegisterRd \neq 0$

sub	\$2, \$1, \$3
and	\$12, \$2, \$5
or	\$13, \$6, \$2
add	\$14, \$2, \$2
sw	\$15, 100(\$2)

Data Forwarding

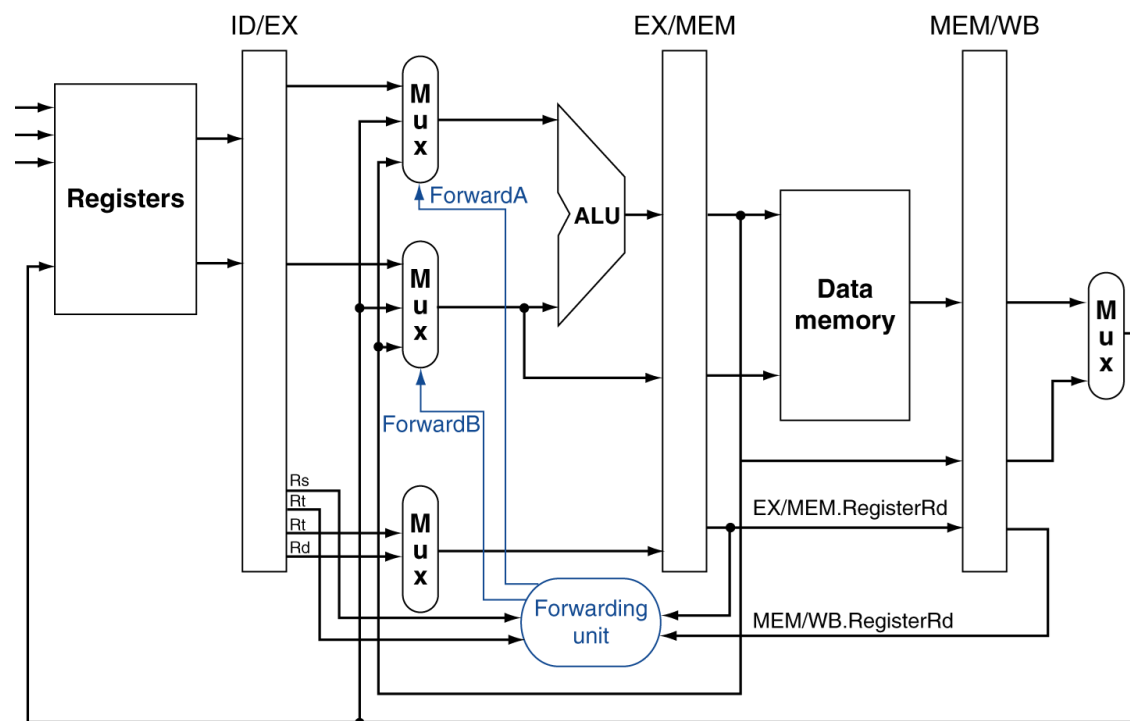
- Allow inputs to ALU not just from ID/EX, but also **later pipeline registers**
- Use **multiplexors** and **control signals** to choose appropriate inputs to ALU



Dependencies between pipelines move forward in time



Forwarding Paths



b. With forwarding



Data Hazard: Detection and Forwarding

- Forwarding unit determines multiplexor control according to the following rules:

1. EX hazard

```
if (    EX/MEM.RegWrite                // if there is a write...  
    and ( EX/MEM.RegisterRd  $\neq$  0 )    // to a non-$0 register...  
    and ( EX/MEM.RegisterRd = ID/EX.RegisterRs ) ) // which matches, then...  
    ForwardA = 10
```

```
if (    EX/MEM.RegWrite                // if there is a write...  
    and ( EX/MEM.RegisterRd  $\neq$  0 )    // to a non-$0 register...  
    and ( EX/MEM.RegisterRd = ID/EX.RegisterRt ) ) // which matches, then...  
    ForwardB = 10
```